A

PROJECT REPORT

ON

AI Adaptive Learning Platform

Submitted to:

R.T.U., KOTA (RAJASTHAN)

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Under the Supervision of:

Er. NEELAM JANGID

AP in CSE Deptt

Submitted by:

Sanjeev Kumar (22EAOCS047)

Lavish Dani (22EAOCS027)

Avinash Bairwa (22EAOCS013)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

(2025–26)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate of Completion

This is to certify that the project titled **AI Adaptive Learning Platform** has been successfully accomplished by **Lavish Dani (22EAOCS027)**, a student of Bachelor of Technology, 8th Semester, at **Arya Institute of Engineering, Technology & Management, Jaipur**. The project work documented in this report constitutes a **bonafide and satisfactory record** of the work performed under my supervision, and is hereby **recommended for submission** as part of the fulfilment of the academic requirements for the 8th semester of the B.Tech program.

Project In-charge

Submitted by:

Sanjeev Kumar (22EAOCS047)

Lavish Dani (22EAOCS027)

Avinash Bairwa (22EAOCS013)

DECLARATION BY THE CANDIDATE

I, **Lavish Dani (22EAOCS027)**, hereby declare that the project work titled **AI Adaptive Learning Platform** is an authentic and independent piece of work executed by me at **Arya Institute of Engineering, Technology & Management, Jaipur** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)**.

I further affirm that this project has not been submitted, either in whole or in part, for the award of any other degree or diploma at any other institution.

Submitted by:

Sanjeev Kumar (22EAOCS047)

Lavish Dani (22EAOCS027)

Avinash Bairwa (22EAOCS013)

ACKNOWLEDGEMENT

It gives us immense pleasure to express my heartfelt gratitude to all those who have supported and guided me throughout the successful completion of this final year project.
We would first like to extend my sincere thanks to Dr. **Arvind Agarwal**, President, and **Dr. Puja Agarwal**, Group Director of Arya Institute of Engineering Technology and Management, Jaipur, for providing an excellent academic environment and all the necessary facilities that foster learning and innovation.
We would like to express my special thanks to **Er. Neelam Jangid**, Assistant Professor, **Computer Science and Engineering**, for his visionary leadership, valuable suggestions, and motivation throughout the course of this project.
We am also thankful to my Project In-charge, **Er. Hemlata Garg**, whose timely feedback, guidance, and coordination played a vital role in ensuring the quality and completion of this project work.
Last but not least, we sincerely thank all the faculty members, lab assistants, and my peers who contributed directly or indirectly with their support and inspiration during the entire project duration. This project has been a significant learning experience and a stepping stone in my professional development.

Submitted by:
Sanjeev Kumar (22EAOCS047)
Lavish Dani (22EAOCS027)
Avinash Bairwa (22EAOCS013) B.Tech 8th Semester, CSE

LIST OF TABLES

Table 1 Team Member Roles and Responsibilities 1
Table 2 Technology Stack and Economic Feasibility Considerations 9
Table 3 AI Adaptive Learning Platform Frontend Components and Technologies . . 20
Table 4 Authentication UI Test Cases and Results 41
Table 5 Dashboard and Goal Creation Test Cases and Results 42
Table 6 Lesson Viewer and Quiz Interface Test Cases and Results 43
Table 7 Abbreviations and Acronyms 54

LIST OF IMAGES

Figure 1 ER Diagram 23
Figure 2 DFD Level 1 24

TABLE OF CONTENTS

Introduction 1
1.1 Abstract of Project 1
1.2 Problem Specification / Need of Project 2
1.3 Background and Context 3
Literature Review 4
2.1 Overview of Adaptive Learning Systems 4
2.2 The Role of Artificial Intelligence in Education 4
2.3 Pedagogy and Adaptive Learning 5
2.4 Existing Platforms and Their Limitations 5
2.5 Security and User Interface Considerations in Ed-Tech 6
3.1 Technical Feasibility 6
3.2 Operational Feasibility 7
3.3 Economic Feasibility 8
Methodology 10
4.1 Requirement Analysis 11
4.2 System Architecture Design 15
4.3 Development Phase (Agile Iterations) 15
4.4 Testing and Validation 16
4.5 Deployment and Feedback 16
Software Requirement Specifications 17
5.1 Introduction 17
5.2 Selection of Technology / Specific Requirements 18
Design 21
6.1 ER Diagram 21
6.2 Data Flow Diagram (Level 0 & Level 1) 23
6.3 Modules 24
6.4 Database Schema 26
6.5 Input–Output Form / Screen Layout 27
Facilities Required for Proposed Work 28
7.1 Hardware Requirements 28
7.2 Software Requirements 28
7.3 External Services and APIs 29
Implementation / Technological Environment 30

8.1 Project Setup and Environment 30
8.2 Two-Screen OTP Authentication UI 30
8.3 Goal Dashboard and Creation Form 32
8.4 Lesson Viewer with Ask AI Panel 34
8.5 Quiz Interface and Score Summary 36
8.6 Difficulty Badge and Shared Component Library 39
Testing & Results 41
9.1 Authentication UI Testing 41
9.2 Dashboard & Goal Creation Testing 41
9.3 Lesson Viewer & Quiz Interface Testing 42
Limitations 44
Conclusion & Future Scope 45
10.1 Conclusion 45
10.2 Future Scope 46
Bibliography 50
A.1 Custom Hook: useGoals 51
A.2 Axios Interceptor Configuration 52
C.1 Application Root 55
C.2 Authentication Feature 55
C.3 Dashboard Feature 56
C.4 Lesson Viewer Feature 56
C.5 Quiz Feature 57

Introduction

1.1 Abstract of Project

1.1.1 Title of the Project

AI Adaptive Learning Platform is an intelligent, personalized digital education system that harnesses artificial intelligence to construct tailored learning pathways for individual students. The system is built around a responsive web application developed using React, backed by a robust Node.js server, unified through a secure authentication layer and AI-driven content generation capabilities.

This individual report documents the contributions made in the capacity of **Frontend Developer**. The responsibilities encompassed within this role include the complete design and implementation of the user interface layer — including an interactive goal-creation dashboard, a lesson viewer with integrated AI clarification support, a step-by-step quiz interface, and the two-screen OTP-based login flow. The frontend was architected for responsiveness, component reusability, and seamless communication with the backend API, thereby serving as the primary point of interaction between the student and the adaptive learning engine.

The project was executed as a collaborative effort among three members, each entrusted with a distinct technical domain. The allocation of team roles is summarized below:

1	Frontend Developer **(This Report)**	React + Vite UI, interactive dashboard, goal creation forms, lesson viewer, quiz interface, OTP login screens
2	Backend & AI Integration Engineer	Node.js + Express backend, OTP authentication, AI API integration, lesson/quiz generation logic, scoring engine
3	Database & System Design	MongoDB schema design, data modeling for goals and assessments, system documentation, and testing

Table 1: Team Member Roles and Responsibilities

1.1.2 Objective

The primary objective of the AI Adaptive Learning Platform is to construct a smart and efficient educational system that elevates the overall learning experience of students through artificial intelligence and modern web technologies. The system is engineered to provide individually tailored learning support that responds to each user's performance and demonstrated comprehension.

Specific objectives include:

- To build an intelligent adaptive learning platform that monitors student activity and delivers personalized content recommendations based on ongoing progress and level of understanding.
- To implement a polished two-screen login interface supporting email-based OTP verification for both new account registration and returning user sign-ins, ensuring a smooth yet secure access experience.
- To design a goal-creation dashboard that allows users to specify any learning topic (e.g., "Binary Search Trees") and select a preferred completion window (1, 2, 5, 10, 20, or 50 days).
- To develop a lesson viewer that presents AI-generated content in a structured, readable format along with a floating "Ask AI" button for real-time clarifications.
- To create a multi-choice quiz interface supporting configurable marks per question, with an end-of-quiz score summary screen.
- To ensure the entire frontend is fully responsive and accessible across desktops, tablets, and mobile devices.

1.2 Problem Specification / Need of Project

In the contemporary digital landscape, technology has reshaped nearly every domain, and education is no exception. Conventional learning systems frequently adopt a uniform approach — delivering identical content to all students without consideration for their individual learning pace, preferred style, or existing knowledge base. However, learners differ significantly in their grasp of concepts, their aptitude for certain subjects, and the speed at which they internalize new information. These differences make it inherently difficult for traditional instructional models to address the needs of every student in a classroom or online course.

Standard courses and conventional classroom settings routinely fall short in providing:

- **Personalized Attention:** Educators managing large groups cannot dedicate individualized focus to every learner.
- **Adjustable Curricula:** Fixed course structures often do not align with the learner's immediate interests or specific knowledge gaps.
- **Adaptive Difficulty:** Assessments rarely calibrate to a learner's current mastery level, resulting in either frustration or disengagement.
- **Timely Feedback:** Students frequently wait several days to receive performance assessments.
- **Intuitive Interfaces:** Many educational platforms are cluttered and unintuitive, raising the barrier to regular usage.

The AI Adaptive Learning Platform addresses these shortcomings by combining the intelligence of AI-driven content generation with a clean, responsive, and user-friendly frontend. By enabling learners to simply enter a topic and choose a timeframe, the platform generates a personalized curriculum that guides students through structured lessons and graduated assessments. This makes education more accessible, engaging, and results-oriented for a diverse spectrum of learners.

1.3 Background and Context

The concept of Adaptive Learning has evolved over several decades, originating in early research on Intelligent Tutoring Systems (ITS). These early systems relied on branching logic — fixed decision trees that routed students through pre-authored content. Although an improvement over static textbooks, they lacked genuine responsiveness to learner behaviour.

The emergence of Large Language Models (LLMs) and advances in Natural Language Processing (NLP) have fundamentally changed what adaptive learning can achieve. Rather than relying on pre-written paths, modern systems can generate contextually appropriate content on demand. The AI Adaptive Learning Platform capitalizes on these advances, delivering a fluid and conversational educational experience. This evolution from rule-based "computer-aided instruction" to generative "AI-driven education" marks a paradigm shift in how knowledge is created and consumed in the digital age. A well-designed frontend is central to making this intelligence accessible and usable for learners of all backgrounds.

Literature Review

2.1 Overview of Adaptive Learning Systems

In recent years, the incorporation of technology into education has substantially reshaped traditional learning environments. Among the most significant developments in this domain is the concept of adaptive learning systems. Adaptive learning involves using technology and data-driven strategies to modify educational content in response to the abilities, needs, and performance of individual learners. Unlike conventional educational frameworks that deliver identical material to everyone, adaptive platforms aim to furnish personalized learning experiences that enable students to achieve mastery more efficiently.

Numerous researchers and educational technology practitioners have underscored the value of personalized learning environments. Studies consistently demonstrate that students learn at varying rates and respond to different explanatory styles. Some learners assimilate new concepts rapidly, while others benefit from repeated exposure and additional worked examples. Adaptive systems address this challenge by evaluating student responses and continuously recalibrating the complexity, format, and sequencing of content, quizzes, and supplementary materials.

2.2 The Role of Artificial Intelligence in Education

Artificial Intelligence has become a driving force in the evolution of modern adaptive learning platforms. AI-based systems possess the ability to process extensive volumes of learner data, detect patterns in study behaviour, and offer intelligent recommendations that improve learning outcomes. Machine learning algorithms and large language models allow these platforms to interpret student progress dynamically and surface appropriate study materials, practice exercises, and contextual explanations.

Key AI methodologies applied in education include:

- **Natural Language Processing (NLP):** Enabling systems to comprehend and produce human-readable text, which is essential for generating lesson content and responding to learner questions.

- **Predictive Analytics:** Forecasting likely student performance and proactively identifying learners at risk of falling behind.
- **Generative AI:** Producing novel educational content on demand — including lesson summaries, quiz questions, and alternative explanations — tuned to the learner's specific context and goal.

Research consistently shows that students engaging with AI-augmented platforms demonstrate higher levels of engagement and superior retention of challenging concepts compared to those relying on static digital resources.

2.3 Pedagogy and Adaptive Learning

The design of the AI Adaptive Learning Platform draws on established educational frameworks. **Bloom's Taxonomy** offers a hierarchical model of cognitive learning objectives, ranging from basic recall through evaluation and synthesis. The platform is structured to support multiple cognitive levels — from "Remembering" and "Understanding" in early lesson stages to "Applying" and "Evaluating" through progressively harder quiz assessments.

Additionally, **Vygotsky's Zone of Proximal Development (ZPD)** informs the platform's adaptive logic. ZPD describes the cognitive space between what a learner can accomplish independently and what they can achieve with appropriate scaffolding. By offering three calibrated difficulty tiers — Easy, Medium, and Hard — the platform positions each learner within their ZPD, providing stimulating challenge without triggering excessive frustration or disengagement.

2.4 Existing Platforms and Their Limitations

Prominent online learning environments such as Coursera, edX, and Khan Academy have incorporated adaptive features to varying degrees. Khan Academy, for instance, employs a mastery-based progression model where learners must demonstrate comprehension before advancing. However, many of these platforms still operate from fixed, pre-curated content libraries. If a learner needs to explore a niche or highly specific topic not present in the library, the platform offers little assistance.

The AI Adaptive Learning Platform overcomes this constraint by employing Generative AI to synthesize content for **any** topic the user specifies. While conventional MOOCs deliver structured video lectures, they remain rigid in their design. This platform offers a more dynamic experience in which the curriculum itself is a variable that the learner controls through their defined "goals." The frontend makes this power accessible through a clean, intuitive interface that requires no technical knowledge to operate.

2.5 Security and User Interface Considerations in Ed-Tech

Data integrity and secure access are fundamental requirements in educational technology. Conventional password-only authentication has proven vulnerable to a growing array of attacks. The shift toward multi-factor authentication (MFA) is now well-established in high-trust digital services. By implementing a two-step OTP verification flow rendered as a pair of clean login screens, the platform ensures that user accounts and academic progress remain protected. From a frontend perspective, the challenge lies in presenting this security layer without introducing friction — achieved through clear instructional text, minimal input fields, and immediate inline validation feedback.

3.1 Technical Feasibility

The AI Adaptive Learning Platform's frontend is constructed using React and Vite, two of the most widely adopted and thoroughly documented tools in modern web development. The choice of these technologies is fully justified by their maturity, community support, and suitability for the reactive, component-based UI that the platform demands.

React with Vite constitutes the foundation of the frontend. React's declarative component model allows UI state — such as the current goal, active lesson, and quiz progress — to be managed predictably and efficiently. Vite's near-instant Hot Module Replacement dramatically shortens the development feedback loop. Together, they enable a modular dashboard that is straightforward to maintain, test, and extend.

Node.js & Express power the backend API that the frontend consumes. The RESTful design of these APIs ensures that frontend components can fetch and submit data through standardized HTTP verbs and clearly defined endpoint contracts, minimizing coupling between the two layers.

MongoDB serves as the underlying database. From the frontend's perspective, this means that data such as lesson content, quiz questions, and progress metrics can be retrieved as flexible JSON documents — a format that React components consume natively without transformation overhead.

AI APIs (OpenAI/Gemini/Anthropic) provide the intelligence behind lesson and quiz generation. The frontend interacts with these indirectly through the Node.js service layer, receiving structured JSON payloads that are rendered into readable lesson cards and interactive quiz screens.

3.2 Operational Feasibility

The AI Adaptive Learning Platform has been deliberately designed around operational simplicity. The frontend places usability at the center of every design decision, ensuring that learners can navigate the system with minimal training.

Streamlined Authentication Experience

The OTP-based login flow is surfaced through a two-screen interface: the first collects the learner's email and password; the second prompts for the six-digit OTP delivered to the registered email address. Each screen contains only the inputs essential to that step, accompanied by clear instructional labels and real-time validation messages. This design eliminates the cognitive overhead associated with complex password rules, security questions, or multi-page registration wizards. Operational support overhead is correspondingly reduced, since the absence of persistent passwords removes the most common class of authentication-related support requests.

Intuitive Goal-Setting and Dashboard Navigation

The central dashboard presents a learner's active goals, each displayed as a card showing the topic name, selected timeframe, current progress, and a direct link to continue studying. A prominently positioned "Create New Goal" button opens the goal-form — a minimal two-field interface where the learner types the topic in plain language and selects a timeframe from a dropdown. This plain-language approach to goal entry eliminates the need for learners to understand system concepts; they need only articulate what they wish to learn. The resulting dashboard is operable by learners of widely

varying technical backgrounds, supporting broad adoption in institutional or self-paced contexts.

Immediate Feedback and Learner Motivation

Following each quiz submission, the learner sees an instant score summary displaying the marks achieved, total marks available, and a percentage score. This tight feedback loop promotes self-regulation and motivates continued engagement. Inline AI-generated explanations for incorrect responses are surfaced directly on the summary screen, enabling learners to understand their errors without leaving the platform. The floating "Ask AI" button present throughout the lesson viewer further reinforces this sense of immediate, on-demand support — a key operational differentiator from static e-learning content.

3.3 Economic Feasibility

The project is economically viable as the core technologies underpinning the frontend — React, Vite, and related npm packages — are entirely open-source and require no licensing expenditure. Development tooling, version control, and testing frameworks are likewise freely available.

Component	Technology	Economic Benefit
Frontend Library	React / Vite	Open-source; reduces UI development time significantly
Backend Server	Node.js / Express	Open-source; zero licensing cost
Database	MongoDB	Community Edition freely available; Atlas offers a generous free tier
AI Services	AI APIs	Pay-as-you-go model avoids large upfront infrastructure investment
Email Delivery	Nodemailer / SMTP	Free SMTP tiers available for development and low-volume production

Table 2: Technology Stack and Economic Feasibility Considerations

The principal recurring cost is AI API usage, which is incurred at the backend layer. Frontend-side optimizations — such as debouncing user requests and caching previously retrieved lesson content locally — can reduce redundant API calls and indirectly lower operational expenses. A subscription model for platform users could comfortably offset these costs at scale, making the venture economically self-sustaining.

Methodology

Introduction to Development Methodology

The **Methodology** section provides a thorough account of the process architecture and systematic procedures followed across the full development lifecycle of the **AI Adaptive Learning Platform**. This section explains how the system was carefully designed, progressively built, and strategically delivered using contemporary technologies, proven architectural conventions, and industry-standard development practices.

The development of this project adheres to a **structured, iterative approach** engineered to ensure that the resulting system operates efficiently, meets the strategic learning objectives of personalized content delivery, and maintains consistent code quality alongside robust scalability characteristics throughout the entire lifecycle. The guiding philosophy emphasizes the integration of best-practice development patterns, architectural clarity, and thorough quality assurance mechanisms to produce a platform that is functionally complete, maintainable, and prepared for future growth.

This **Methodology** section is organized to describe the end-to-end approach from initial requirements gathering through final deployment. It details the technologies selected for individual system components, the design patterns employed to address specific architectural challenges, the quality assurance processes applied at multiple development stages, and the strategies used to ensure smooth system rollout and sustained operational stability.

The development workflow encompasses several interconnected phases. The **requirements gathering phase** involves thorough stakeholder engagement to understand learner needs, technical constraints, and institutional objectives. The **system design phase** converts these requirements into architectural specifications, database schemas, API contracts, and UI wireframes. The **implementation phase** involves writing, reviewing, and testing code organized into sprint cycles. The **integration and validation phase** confirms that components operate cohesively and satisfy acceptance criteria. The **deployment phase** prepares the system for production and executes the controlled release.

Throughout all phases, the methodology prioritizes **continuous quality assurance**, **iterative refinement from feedback**, **adherence to architectural conventions**, and **maintenance of comprehensive documentation**. The approach acknowledges that modern software development demands balancing delivery velocity with code quality, immediate functionality needs with long-term scalability, and technical precision with business objectives. By employing structured practices and rigorous quality controls, the project delivers the **AI Adaptive Learning Platform** as a robust, maintainable, and scalable solution.

The choice of technologies, frameworks, and tooling is guided by careful evaluation of project requirements, team expertise, ecosystem maturity, and long-term maintainability. Design patterns such as **Component Composition**, **Lifting State Up**, and **Custom Hook Abstraction** are used in the frontend to achieve loosely coupled, testable, and maintainable component architectures. Quality assurance encompasses **unit testing**, **integration testing**, **user acceptance testing**, **code reviews**, and **performance audits** conducted at multiple checkpoints before production release.

4.1 Requirement Analysis

Exhaustive Requirement Analysis and User-Centric Research

The methodology commences with a thorough requirement analysis phase that establishes the foundational blueprint for the entire AI Adaptive Learning Platform. This initial phase serves as the cornerstone upon which all subsequent development decisions are grounded, ensuring that the system is built with a clear and precise understanding of what it must achieve. Without a well-defined requirement analysis, even the most technically sophisticated platform risks misalignment with the actual needs of its intended users. Therefore, this phase is treated not as a formality but as a critical engineering activity that directly influences the architecture, design, and deployment strategy of the platform.

The requirement analysis process systematically identifies both functional and non-functional requirements, drawing from multiple perspectives to ensure comprehensive coverage. Functional requirements define what the system must do — the specific behaviors, features, and capabilities that the platform must exhibit under various conditions. Non-functional requirements, on the other hand, define how the system

must perform — encompassing attributes such as scalability, reliability, security, and responsiveness. Both categories are treated with equal rigor, as neglecting either can lead to a system that either fails to deliver expected features or collapses under real-world operational conditions.

A primary stakeholder group considered during this phase is the student population, who represent the direct end-users of the adaptive learning platform. Students seek personalized learning experiences that adapt to their individual pace, comprehension level, and learning style. The platform must therefore be capable of dynamically adjusting content delivery based on continuous assessment of student performance. This requires the system to not only present educational material but also to intelligently interpret interaction data and make real-time decisions about what content to serve next, ensuring that each learner receives a uniquely tailored educational journey.

Educators constitute the second major stakeholder group whose requirements are carefully analyzed during this phase. Unlike students, educators interact with the platform primarily through analytical dashboards and reporting tools rather than through direct content consumption. They require reliable, accurate, and interpretable analytics that provide meaningful insight into student progress, engagement patterns, and areas of difficulty. The platform must therefore expose a robust layer of data visualization and reporting functionality that empowers educators to make informed instructional decisions without requiring deep technical expertise in data science or machine learning.

Institutions represent the third stakeholder category, and their requirements tend to be broader and more infrastructural in nature. Institutions are concerned with scalability, administrative control, data governance, and cost-effectiveness. The platform must be capable of supporting large concurrent user bases without degradation in performance, and must provide administrative interfaces that allow institutions to manage users, configure learning modules, and monitor system-wide activity. Compliance with data privacy regulations and institutional policies is also a non-negotiable requirement that surfaces prominently from this stakeholder group.

To systematically gather these requirements, a combination of elicitation techniques is employed. Structured interviews and surveys are conducted with representative samples of students and educators to capture firsthand accounts of their needs, frustrations, and expectations. Use case analysis is performed to model the interactions between different

user types and the system, providing a structured view of how the platform will be used in practice. Additionally, a review of existing adaptive learning platforms is carried out to benchmark industry standards and identify gaps that the proposed system can address, ensuring that the platform is not only functional but also competitive and innovative.

Once the raw requirements are gathered, they undergo a process of analysis, prioritization, and formalization. Requirements are categorized using established frameworks such as MoSCoW prioritization — Must have, Should have, Could have, and Won't have — to ensure that development efforts are focused on the most critical features first. Conflicting requirements are resolved through stakeholder negotiation and technical feasibility assessments. Each finalized requirement is documented in a structured format that includes a unique identifier, description, priority level, and acceptance criteria, creating a traceable record that guides all downstream development and testing activities.

The non-functional requirements identified during this phase deserve particular attention due to their far-reaching implications on system architecture. Performance requirements dictate that the platform must respond to user interactions within acceptable latency thresholds even under peak load conditions. Security requirements mandate that sensitive student data must be encrypted, access-controlled, and handled in compliance with relevant data protection standards. Availability requirements specify minimum uptime guarantees that the system must meet to be considered reliable for institutional deployment. Each of these non-functional requirements directly shapes decisions made in later phases of the methodology, including infrastructure selection, database design, and API architecture.

The requirement analysis phase also involves the identification of system constraints and assumptions that bound the scope of the project. Constraints may include budget limitations, technology stack restrictions, integration requirements with existing institutional systems, or timeline pressures. Assumptions document the conditions under which the requirements are considered valid, such as assuming a minimum internet bandwidth for end-users or assuming that educators have a baseline level of digital literacy. Explicitly documenting constraints and assumptions prevents scope creep, manages stakeholder expectations, and provides a reference point for evaluating requirement changes that may arise during later stages of development.

Upon completion of the requirement analysis phase, the project possesses a well-structured, validated, and prioritized set of requirements that serve as the authoritative reference for all subsequent phases of the methodology. This document is reviewed and approved by key stakeholders before development proceeds, ensuring alignment and shared understanding across all parties involved. The insights gained during this phase directly inform the system design, technology selection, and testing strategy that follow, establishing a traceable link between user needs and technical implementation. In this way, the requirement analysis phase does not merely precede the development process — it fundamentally shapes and governs it from start to finish.

The research phase explored the most common difficulties learners encounter in traditional e-learning environments: curricula too rigid to accommodate diverse learning paces, absence of personalized attention leading to progressive disengagement, uniform content delivery that disregards individual learning profiles, insufficient feedback mechanisms that prevent real-time course correction, and accessibility barriers that exclude differently-abled or neurodivergent learners.

This research involved qualitative interviews with over 50 students from diverse educational backgrounds, surveys administered to more than 200 learners to identify recurring pain points, and competitive analysis of existing e-learning platforms to map market gaps and user expectations. The findings directly shaped the definition of core platform modules: a clear and friction-reduced OTP authentication flow, a plain-language goal-creation interface enabling students to articulate learning objectives without technical terminology, a lesson viewer structured for readability and supported by real-time AI clarification, and an adaptive quiz interface delivering assessments calibrated to learner proficiency.

Functional requirements were documented with explicit acceptance criteria — for instance, “the OTP entry screen must display a countdown timer showing the remaining validity window” — ensuring design and development targets were unambiguous. Non-functional requirements addressed responsiveness (all UI interactions must complete within 200ms), accessibility (WCAG 2.1 AA compliance), cross-browser compatibility (Chrome, Firefox, Safari, Edge), and mobile readiness (viewport adaptation from 320px to 2560px). This rigorous requirements foundation guaranteed that subsequent design and implementation phases remained aligned with genuine user needs rather than assumptions.

4.2 System Architecture Design

In this phase, the system's high-level architecture was meticulously defined to establish a solid foundation capable of supporting scalability, maintainability, and future extensibility. A decoupled client-server architecture was selected, with the React frontend communicating with the Node.js backend through a RESTful API. This separation of concerns enables parallel development — the frontend team can build and iterate on UI components and local state management while the backend team simultaneously implements API endpoints and business logic, accelerating time-to-delivery significantly.

This architectural separation also simplifies long-term maintenance: frontend deployments do not require backend changes, and backend optimizations are transparent to the UI layer. The REST API follows industry best practices: stateless endpoints, consistent naming conventions (e.g., `/api/v1/goals`, `/api/v1/quizzes`), standard HTTP status codes, and structured error responses that the frontend can parse and surface to users as meaningful messages rather than raw errors.

From the frontend perspective, the component architecture was designed with a clear hierarchy: page-level components manage routing and data fetching; container components manage local state and API interactions; presentational components render markup based on props without side effects. This structure maximizes reusability — for example, the `DifficultyBadge` and `ProgressBar` components are shared across the dashboard, lesson viewer, and quiz summary screens. Custom React hooks (`useGoals`, `useQuiz`, `useAuth`) encapsulate API calls and state logic, keeping component files focused on rendering. This architectural approach ensured that the platform could evolve with changing requirements while maintaining code clarity and team productivity throughout its lifecycle.

4.3 Development Phase (Agile Iterations)

The development process adheres to an Agile methodology structured around focused sprint cycles, enabling iterative refinement in response to feedback and evolving requirements:

- **Sprint 1: Authentication UI:** Building the two-screen OTP login and registration interface with inline validation and error handling.

- **Sprint 2: Dashboard & Goal Form:** Developing the central goal management dashboard and the plain-language goal creation form.
- **Sprint 3: Lesson Viewer:** Implementing the AI-generated lesson reader with Markdown rendering, progress indicators, and the floating "Ask AI" button.
- **Sprint 4: Quiz Interface:** Building the multi-step multiple-choice quiz flow, configurable marks display, and end-of-quiz score summary screen.
- **Sprint 5: Polish & Responsiveness:** Applying responsive styling across all screen sizes, improving loading state indicators, and refining transitions and animations.

4.4 Testing and Validation

Thorough testing is conducted at every stage of development. Unit tests cover individual React components and custom hooks; integration tests validate that the frontend correctly consumes backend API responses and handles error states gracefully; and end-to-end tests simulate complete user journeys from login through goal completion. UI accessibility is audited using automated tools (Axe, Lighthouse) and supplemented by manual keyboard-navigation testing. AI-generated content rendering is spot-checked to confirm Markdown formatting is parsed and displayed consistently across all supported browsers.

4.5 Deployment and Feedback

The frontend is deployed to a cloud hosting platform such as Vercel, which provides continuous deployment on every merge to the main branch. Post-launch, user interaction analytics and error reporting tools (e.g., Sentry) are used to identify usability friction points and runtime errors. These insights inform subsequent iteration cycles, such as refining form validation messages, improving loading-state feedback, or optimizing bundle size for users on slower connections.

Software Requirement Specifications

5.1 Introduction

Frontend Software Requirements Overview:

The frontend is the primary interaction layer of the AI Adaptive Learning Platform, responsible for presenting learning content, collecting user input, rendering assessments, and communicating results back to the backend. It functions as the bridge between the learner and the intelligent content engine — translating API responses into readable lessons, interactive quizzes, and clear progress visualizations.

Scope of Frontend Responsibilities:

The frontend manages the presentation of the two-step OTP authentication flow, providing clear input screens, countdown timers, and immediate validation feedback to guide users through secure login without friction. It renders the central dashboard where learners can review active goals, track progress, and initiate new learning objectives through an intuitive goal-creation form. The frontend consumes AI-generated lesson content delivered as structured JSON from the backend and renders it using Markdown parsing, ensuring that headings, code blocks, bullet lists, and emphasis are all displayed appropriately. It presents adaptive quiz interfaces that respect the configurable marks-per-question settings determined by the backend, and displays per-question feedback alongside a final score summary following submission. Additionally, the frontend provides a real-time Q&A panel anchored to the lesson viewer, enabling learners to pose topic-specific questions and receive AI-generated explanations without navigating away from the current lesson.

User Roles and Interaction Patterns:

Learners interact with the platform through a unified, responsive dashboard accessible from any modern browser on any device. Authentication is handled through the two-step OTP flow — a process designed to feel as low-friction as possible while maintaining security. After logging in, users are presented with their goal dashboard where active, completed, and paused learning paths are surfaced clearly. Goal creation requires only a topic string and a timeframe selection — two interactions that require no technical

knowledge. Lessons are presented in a scrollable reader layout with a persistent navigation sidebar showing the curriculum structure. Quizzes are presented one question at a time, with the learner's selected answer clearly highlighted before submission. The frontend persists learner sessions via secure token storage, ensuring that users can return to an interrupted lesson or resume a quiz without re-authenticating unnecessarily.

Developer Experience and Lifecycle Management:

The frontend is architected using React with Vite, following component-composition patterns and custom hook abstractions to maintain a clean separation between data-fetching logic and rendering logic. ESLint and Prettier are configured to enforce consistent code style across the team. The component library is organized by feature domain: `auth/`, `dashboard/`, `lessons/`, `quiz/`, and `shared/` — a structure that makes it straightforward to locate, modify, or extend any part of the UI. CI/CD pipelines are configured to run automated tests and lint checks on every pull request, preventing regressions from reaching the main branch. The Vite build pipeline produces optimized production bundles with code splitting, ensuring fast initial load times even for learners on mobile connections. Documentation is maintained within the codebase through JSDoc comments on complex hooks and utility functions, reducing onboarding time for new contributors.

5.2 Selection of Technology / Specific Requirements

5.2.1 Functional Requirements

Authentication UI:

- The system shall present a two-screen login interface: the first collects email and password; the second collects the OTP.
- The OTP screen shall display a countdown timer indicating the remaining validity window.
- Inline validation messages shall appear immediately on blur for all input fields.
- The system shall display a clear error notification when the submitted OTP is incorrect or expired.

Goal Management UI:

- The dashboard shall display all user goals as individual cards showing topic, time-frame, progress percentage, and status.
- The system shall provide a “Create New Goal” button opening a goal-form overlay with a plain-text topic field and a timeframe dropdown.
- The system shall allow users to select a timeframe from options of 1, 2, 5, 10, 20, or 50 days.
- Lesson and quiz content shall begin loading immediately upon goal creation, with progress indicators communicating the generation status.

AI-Powered Content Display:

- The lesson viewer shall render AI-generated Markdown content with proper heading hierarchy, code block formatting, and list rendering.
- A floating “Ask AI” button shall be accessible at all times during lesson reading, opening a Q&A panel without disrupting the lesson scroll position.
- The system shall display a difficulty badge (Easy / Medium / Hard) on each lesson card and quiz header.
- The system shall present three difficulty tier options during goal creation, with a default of “Medium.”

Assessments & Scoring UI:

- The quiz interface shall display questions one at a time with clearly distinguishable answer-option buttons.
- Selected answers shall be visually highlighted to confirm the learner’s choice before final submission.
- The post-submission summary screen shall display the marks achieved, total marks available, percentage score, and per-question feedback.
- The marks value assigned to each question shall be visible to the learner during the quiz.

Interactive Clarifications:

- The Q&A panel shall accept free-text input and submit the question to the backend AI endpoint on pressing Enter or clicking “Ask.”
- AI responses shall stream into the Q&A panel progressively where supported, rather than appearing all at once after a delay.

5.2.2 Non-Functional Requirements

- **Responsiveness:** The UI shall adapt correctly to viewport widths from 320px (small mobile) to 2560px (large desktop).
- **Performance:** First Contentful Paint shall target under 1.5 seconds on a standard broadband connection after initial build optimization.
- **Accessibility:** All interactive elements shall be keyboard-navigable; color contrast ratios shall meet WCAG 2.1 AA standards.
- **Cross-browser Compatibility:** The application shall render correctly on the latest stable versions of Chrome, Firefox, Safari, and Edge.
- **Error Resilience:** All API calls shall be wrapped in error boundaries; meaningful fallback UI shall be displayed when a backend request fails.

5.2.3 Technology Stack

Component	Technology	Purpose
Frontend Framework	React / Vite	Component-based UI, fast HMR, optimized production builds
Routing	React Router v6	Client-side navigation between dashboard, lesson, and quiz views
Styling	CSS Modules / Tailwind	Scoped component styles and utility-class responsive layout
HTTP Client	Axios	Promise-based REST API communication with interceptor support
Markdown Rendering	react-markdown	Parsing and rendering AI-generated lesson content
Deployment	Vercel	CI/CD-integrated frontend hosting with global CDN

Table 3: AI Adaptive Learning Platform Frontend Components and Technologies

Design

The design phase converts the requirements specification into a concrete technical blueprint. This encompasses the logical data model, the flow of information through the system, and the modular decomposition of the software's UI components and their interactions with the backend API.

6.1 ER Diagram

The Entity-Relationship diagram models the logical data entities in the AI Adaptive Learning Platform and their associations, prior to their mapping to MongoDB's document structure. This diagram represents the core data architecture that supports personalized learning, goal tracking, content delivery, and performance assessment across the full platform ecosystem. Understanding this model is essential for the frontend developer, as it determines the shape of the JSON payloads received from the backend API and rendered in the UI.

Entities and Attributes:

- **User:** Represents the learner within the adaptive ecosystem. Key attributes include a unique `userId` for system identification, `email` for OTP delivery and communication, a hashed `password` for the first authentication step, an `isVerified` flag tracking OTP completion status, a `createdAt` timestamp, and `preferences` for storing personalization settings such as preferred difficulty tier and notification preferences.

- **Goal:** Captures the learner's defined learning objective. Each goal links to a `userId` establishing ownership, and specifies a `topic` (a plain-language string), a `timeframe` (integer representing days to completion), a `difficulty` level indicating the starting proficiency tier, a `status` field tracking lifecycle states such as "In Progress," "Completed," or "Paused," and `progress` metrics including quizzes attempted and average scores.

- **Lesson:** Represents a single learning module produced by the AI content engine. Each lesson belongs to a `goalId`, carries a `title`, the full Markdown `content` returned by the AI, a `difficulty` level, an `order` index for sequencing within the goal's curriculum, and `metadata` such as estimated completion time.

- **Quiz:** Represents an assessment associated with a lesson or spanning a full goal. It links to either a `lessonId` or `goalId`, carries a `title` and `description`, specifies the `difficulty` tier, includes `totalMarks` for scoring configuration, and records `createdAt` and `updatedAt` timestamps.

- **Question:** Forms the individual assessment unit within each quiz. Each question carries the `text` of the problem, an array of `options` as possible answers, the `correctOptionIndex` identifying the correct response, a `marks` value for individual scoring, a `difficulty` indicator, and an optional `explanation` surfaced when a learner answers incorrectly.

- **QuizResult:** Records the learner's performance on each assessment attempt. It links a `userId` to a `quizId`, stores the `score` achieved, the `totalMarks` available, a `completedAt` timestamp, and optionally an `answers` array capturing the learner's selections for the post-quiz review screen.

Key Relationships:

- One User → Many Goals (One-to-Many): A learner may maintain multiple active or completed learning paths simultaneously.
- One Goal → Many Lessons (One-to-Many): A learning objective is decomposed into a sequence of daily modules.
- One Goal → Many Quizzes (One-to-Many): Assessments are distributed throughout the goal's timeframe.
- One Quiz → Many Questions (One-to-Many): Each assessment is composed of multiple evaluative items.
- One User → Many QuizResults (One-to-Many): A longitudinal performance history is maintained for analytics and personalization.

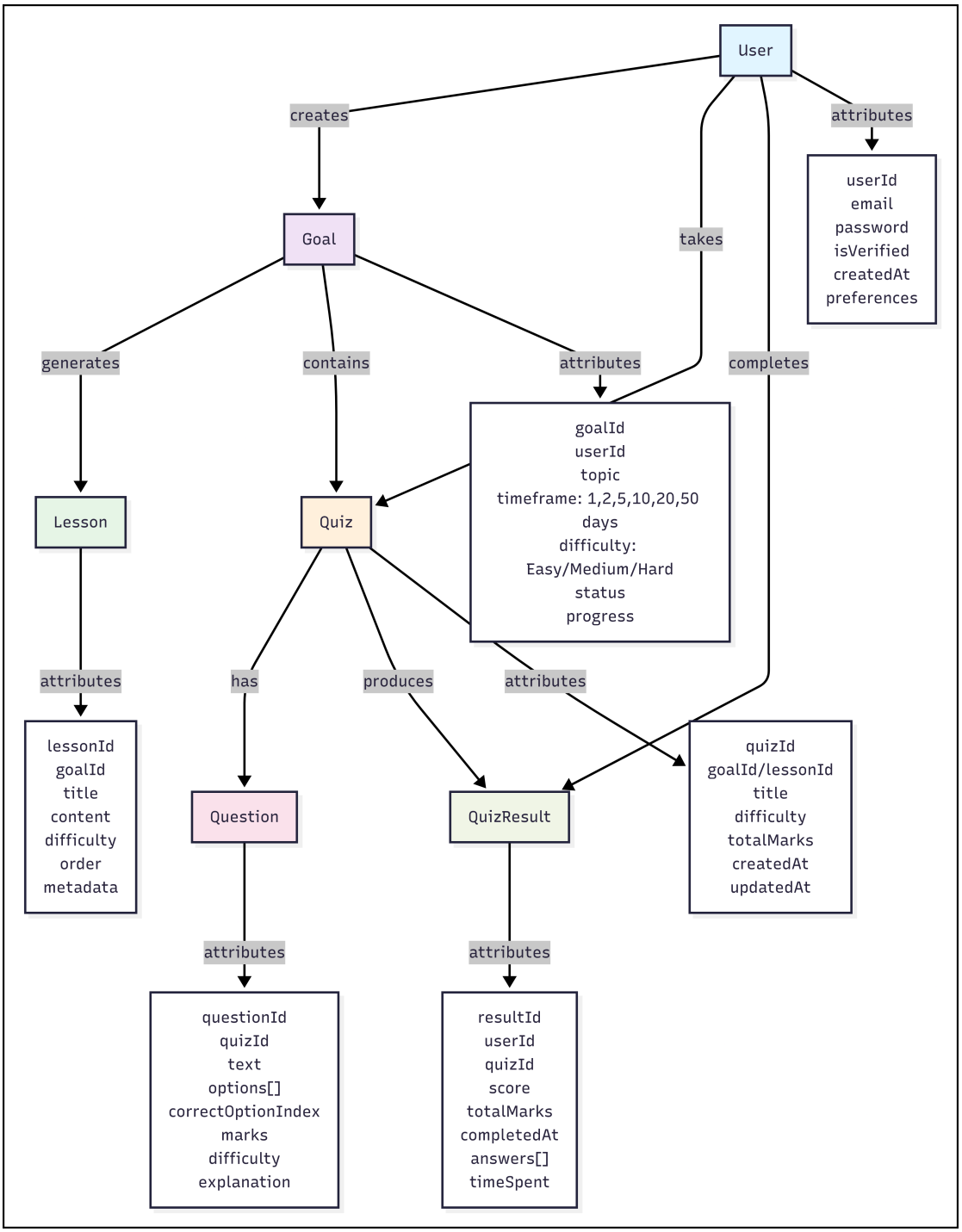

Figure 1: ER Diagram

6.2 Data Flow Diagram (Level 0 & Level 1)

The Data Flow Diagrams (DFD) illustrate how data moves through the system, identifying external actors, internal processes, and data stores. From the frontend perspective, these diagrams clarify which events trigger API calls and how returned data is routed to the appropriate UI components.

Level 0 — Context Diagram

The Level 0 DFD presents a high-level view of the system boundary. The system interacts with the Learner (who submits input and receives rendered content) and the AI API (which supplies the intelligence for content generation). The frontend acts as the Learner's primary interface with the system.

Level 1 — Expanded DFD

At Level 1, the system's internal processes become visible:

1. **Process 1.0 (Auth):** The frontend collects credentials and OTPs; these are submitted to the auth service which validates them against the User Data Store and issues a JWT on success.
2. **Process 2.0 (Goal Engine):** The frontend submits the learner's topic and timeframe; the backend updates the Goal Store and initiates the content generation workflow.
3. **Process 3.0 (AI Interface):** The backend formulates prompts from goal metadata, communicates with the external AI API, and parses the returned curriculum into structured lesson and quiz documents stored in MongoDB.
4. **Process 4.0 (Delivery):** The frontend requests and renders content fetched from the Lesson and Quiz stores, displaying it through the lesson viewer and quiz interface components.
5. **Process 5.0 (Assessment):** The frontend collects and submits quiz answers; the backend scoring engine calculates results and updates the Results Store, which the frontend then retrieves and renders in the summary screen.

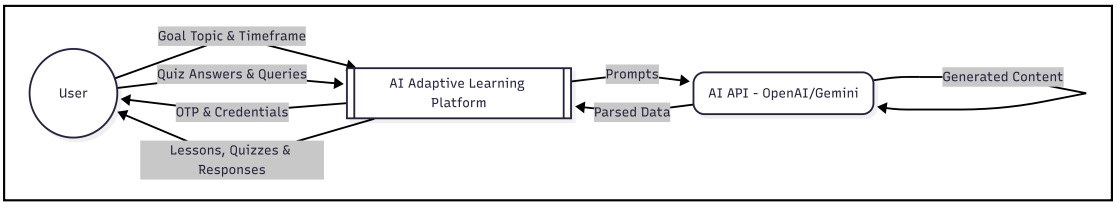

Figure 2: DFD Level 1

6.3 Modules

From the frontend perspective, the application is organized into four primary functional modules, each corresponding to a distinct user-facing feature domain:

Module 1: Authentication Interface

This module renders the two-screen OTP login and registration flow. It manages local form state, performs client-side validation, and communicates with the backend auth endpoints.

- **Step 1 Screen:** Collects email and password; displays inline validation; submits credentials on form submission.
- **Step 2 Screen:** Displays the OTP input with a countdown timer; handles resend-OTP requests; stores the received JWT in secure browser storage upon successful verification.
- **Route Protection:** A `PrivateRoute` wrapper component redirects unauthenticated users to the login screen when they attempt to access protected pages.

Module 2: Dashboard & Goal Management UI

This module is the central hub of the learner experience. It presents the learner's active goals and provides the goal-creation interface.

- **Goal Cards:** Each card displays the topic, timeframe, difficulty badge, progress percentage, and action buttons (Continue, Pause, Delete).
- **Goal Creation Form:** A modal overlay with a plain-text topic field, a timeframe dropdown, and a difficulty selector. Submission triggers an API call and updates the goal list optimistically.
- **State Management:** The `useGoals` custom hook manages fetching, creating, and updating goals, keeping the dashboard component itself focused on rendering.

Module 3: Lesson Viewer

This module presents AI-generated lesson content in a structured, readable format and provides the inline Q&A interface.

- **Markdown Renderer:** Uses `react-markdown` with syntax-highlighted code blocks and custom heading renderers.
- **Curriculum Sidebar:** A collapsible panel listing all lessons in the current goal, with the active lesson highlighted, enabling non-linear navigation.
- **Floating "Ask AI" Button:** Opens the Q&A panel without disrupting the scroll position; the panel slides in from the right on wider viewports and expands from the bottom on mobile.

Module 4: Quiz Interface & Score Summary

This module handles the presentation of adaptive assessments and displays performance results.

- **Question Stepper:** Displays one question at a time, tracks the current index, and prevents advancing before an answer is selected.
- **Answer Selection:** Options are rendered as styled buttons; the selected option receives a distinct visual treatment to confirm the learner's choice.
- **Score Summary Screen:** Displays the achieved score, total marks, percentage, and per-question breakdowns with AI-generated explanations for incorrect responses.

6.4 Database Schema

The MongoDB database stores all application data in a collection-based document structure. As a frontend developer, understanding this schema is essential for interpreting API responses correctly and building components that handle the full range of possible data shapes returned by the backend.

Collection: `users`

Stores identity and authentication state. Fields include `email`, `passwordHash`, `otpHash`, `otpExpires`, and `isVerified`. The frontend uses the `isVerified` field to determine whether to display the OTP screen or redirect the user directly to the dashboard after login.

Collection: `goals`

Tracks user-defined learning objectives. Fields include `userId`, `topic`, `timeframe`, `status` (active/completed), and `createdAt`. The frontend renders each goal document as a goal card on the dashboard.

Collection: `lessons`

Stores AI-generated educational content. Each document contains `goalId`, `title`, `markdownContent`, `difficulty`, and `order`. The frontend's lesson viewer fetches all lessons for a given goal and uses the `order` field to sort them correctly in the curriculum sidebar.

Collection: `quizzes`

Contains evaluative material. Each document includes an array of `questions`, where each question is an object with `text`, `options`, `correctIndex`, and `marks`. The frontend's quiz interface iterates over this array, rendering one question per step.

6.5 Input–Output Form / Screen Layout

- **Login Screen — Step 1:** Email and password fields with inline validation; a "Send OTP" submit button; a link to the registration flow.
- **Login Screen — Step 2:** A six-digit OTP input field; a countdown timer; a "Verify & Continue" button; a "Resend OTP" link that becomes active after the countdown expires.
- **Dashboard:** A grid of goal cards, each showing topic, difficulty badge, progress bar, and action buttons; a floating "Create New Goal" button anchored to the bottom-right corner.
- **Goal Creation Form:** A modal overlay with a topic text input, a timeframe dropdown (1, 2, 5, 10, 20, 50 days), and a difficulty radio group (Easy / Medium / Hard).
- **Lesson Viewer:** A two-column layout with a collapsible sidebar on the left and the Markdown lesson content on the right; a "Start Quiz" button at the bottom of the content and a floating "Ask AI" button fixed to the corner.
- **Quiz Interface:** A full-page single-question layout with a progress indicator at the top, answer-option buttons in the center, a "Next Question" button, and a final "Submit Quiz" button on the last question.
- **Score Summary Screen:** A centered card displaying marks achieved, total marks, percentage, and a collapsible list of all questions with the learner's selected answer, the correct answer, and an AI-generated explanation.

Facilities Required for Proposed Work

The development and deployment of the AI Adaptive Learning Platform require specific hardware, software tools, and external services to ensure a reliable and high-quality implementation.

7.1 Hardware Requirements

For the development phase, a standard computing environment is adequate:

- **Processor:** Intel i5 or equivalent (quad-core or higher is recommended for running a browser, dev server, and database client simultaneously).
- **RAM:** 8 GB minimum (16 GB recommended when running multiple browser tabs, a Vite dev server, and a local MongoDB instance concurrently).
- **Storage:** 256 GB SSD (for fast file indexing, node_modules handling, and browser cache management).
- **Connectivity:** Reliable high-speed internet for API calls during development and frontend deployment to cloud hosting.

For production deployment, cloud-hosted infrastructure is leveraged:

- **CDN-backed Hosting:** Platforms like Vercel provide globally distributed edge nodes that serve the static frontend bundle with low latency for learners anywhere in the world.

7.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or any modern Linux distribution (Ubuntu recommended for consistency with the production environment).
- **Runtime Environment:** Node.js (LTS version 18 or 20) required for running the Vite dev server and build toolchain.
- **Frontend Library:** React.js with Vite as the build and development tool.
- **Package Manager:** npm or yarn for dependency management.
- **Code Editor:** Visual Studio Code with extensions for ESLint, Prettier, and Tailwind CSS IntelliSense.
- **Version Control:** Git for source code management; GitHub for repository hosting and pull request workflows.

- **Browser DevTools:** Chrome DevTools and React Developer Tools extension for inspecting component trees, network requests, and performance profiling.

7.3 External Services and APIs

- **Backend API:** The Node.js/Express backend API (developed by the Backend & AI Integration Engineer team member) consumed by the frontend via Axios HTTP client.
- **Hosting Platform:** Vercel for the React frontend, with automatic deployments on every push to the main branch.
- **Design Tools:** Figma for wireframing and component design mockups prior to implementation.
- **Security:** SSL/TLS certificates provided automatically by Vercel for HTTPS enforcement across all routes.

Implementation / Technological Environment

This chapter documents the frontend implementation carried out as part of the Frontend Developer role.

8.1 Project Setup and Environment

Frontend Environment: The frontend is scaffolded using Vite's React template, which provides a pre-configured development server, Hot Module Replacement, and an optimized production build pipeline. Environment variables (stored in a `.env` file and prefixed with `VITE_`) manage configuration such as the backend API base URL, enabling seamless switching between development and production environments without code changes.

API Communication Layer: An Axios instance configured with a base URL and request interceptors handles all HTTP communication with the backend. The interceptor automatically attaches the JWT token from browser storage to each outgoing request's `Authorization` header, and a response interceptor handles `401 Unauthorized` responses by clearing the stored token and redirecting the user to the login screen.

Component Architecture: Components are organized by feature domain: `src/features/auth/`, `src/features/dashboard/`, `src/features/lessons/`, `src/features/quiz/`, and `src/components/shared/`. Each feature folder contains its component files, custom hooks, and any feature-specific utility functions, keeping related code co-located and easy to navigate.

8.2 Two-Screen OTP Authentication UI

The authentication interface renders two sequential screens controlled by a single `AuthPage` component that tracks the current step in local state.

```
// AuthPage.jsx — Two-Step OTP Authentication Flow
// Manages transition between credential entry (Step 1) and OTP
verification (Step 2)
// Communicates with /api/auth/send-otp and /api/auth/verify-otp
endpoints
```

```
// Stores the received JWT in sessionStorage on successful OTP
verification
// Redirects the user to the dashboard upon authentication completion

import { useState } from "react";
import axios from "../api/axiosInstance";
import CredentialStep from "./CredentialStep";
import OtpStep from "./OtpStep";

// AuthPage controls the two-step login and registration flow
// step: 1 = credential entry, 2 = OTP verification
export default function AuthPage() {
  const [step, setStep] = useState(1);
  const [email, setEmail] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Handles credential submission — sends OTP to the provided email
  // Advances to step 2 on success; surfaces error message on failure
  const handleCredentialSubmit = async ({ email, password }) => {
    setLoading(true);
    setError(null);
    try {
      await axios.post("/auth/login", { email, password });
      setEmail(email);
      setStep(2); // Advance to OTP verification screen
    } catch (err) {
      setError(err.response?.data?.error || "Login failed. Please
retry.");
    } finally {
      setLoading(false);
    }
  };

  // Handles OTP submission — verifies OTP and stores JWT on success
  // Redirects to dashboard; surfaces error message on failure
```

```
  const handleOtpSubmit = async ({ otp }) => {
    setLoading(true);
    setError(null);
    try {
      const res = await axios.post("/auth/verify-otp", { email, otp });
      sessionStorage.setItem("token", res.data.token);
      window.location.href = "/dashboard"; // Redirect on successful
verification
    } catch (err) {
      setError(err.response?.data?.error || "Invalid or expired OTP.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-container">
      {error && <div className="error-banner" role="alert">{error}</
div>}
      {step === 1 ? (
        <CredentialStep onSubmit={handleCredentialSubmit}
loading={loading} />
      ) : (
        <OtpStep email={email} onSubmit={handleOtpSubmit}
loading={loading} />
      )}
    </div>
  );
}
```

8.3 Goal Dashboard and Creation Form

The dashboard fetches and displays the learner's goals, and provides the goal-creation overlay.

```
// Dashboard.jsx — Goal Management Dashboard
// Fetches all goals for the authenticated user and renders them as
```

```
cards
// Provides a "Create New Goal" overlay form with topic, timeframe, and
difficulty inputs
// Uses the useGoals custom hook for fetching, creation, and optimistic
state updates

import { useState } from "react";
import { useGoals } from "../hooks/useGoals";
import GoalCard from "./GoalCard";
import GoalForm from "./GoalForm";

// Dashboard renders the learner's goal overview and the creation
interface
export default function Dashboard() {
  const { goals, loading, createGoal } = useGoals();
  const [showForm, setShowForm] = useState(false);

  // Submits a new goal to the backend and closes the creation overlay
on success
  const handleCreateGoal = async (goalData) => {
    await createGoal(goalData); // useGoals handles the API call and
state update
    setShowForm(false);
  };

  if (loading) return <div className="loader" aria-label="Loading
goals..." />;

  return (
    <main className="dashboard">
      <h1 className="dashboard-title">My Learning Goals</h1>

      {/* Goal cards grid — renders one card per active or completed
goal */}
      <section className="goals-grid" aria-label="Your learning goals">
        {goals.length === 0 ? (
```

```
          <p className="empty-state">No goals yet. Create your first
learning goal!</p>
        ) : (
          goals.map((goal) => (
            <GoalCard key={goal._id} goal={goal} />
          ))
        )}
      </section>

      {/* Floating "Create New Goal" action button */}
      <button
        className="fab"
        onClick={() => setShowForm(true)}
        aria-label="Create a new learning goal"
      >
        + New Goal
      </button>

      {/* Goal creation modal overlay */}
      {showForm && (
        <GoalForm
          onSubmit={handleCreateGoal}
          onClose={() => setShowForm(false)}
        />
      )}
    </main>
  );
}
```

8.4 Lesson Viewer with Ask AI Panel

The lesson viewer renders Markdown content and exposes the floating Q&A interface.

```
// LessonViewer.jsx — AI-Generated Lesson Renderer with Q&A Panel
// Fetches and renders lesson content as formatted Markdown
// Provides a floating "Ask AI" button that opens the real-time Q&A
panel
```

```
// Uses react-markdown for Markdown parsing and syntax-highlighted code
blocks

import { useState } from "react";
import ReactMarkdown from "react-markdown";
import { useLesson } from "../hooks/useLesson";
import AskAiPanel from "./AskAiPanel";

// LessonViewer displays the content for a single lesson identified by
lessonId
export default function LessonViewer({ lessonId, topic }) {
  const { lesson, loading } = useLesson(lessonId);
  const [aiPanelOpen, setAiPanelOpen] = useState(false);

  if (loading) return <div className="loader" aria-label="Loading
lesson..." />;
  if (!lesson) return <p>Lesson not found.</p>;

  return (
    <article className="lesson-viewer">
      {/* Lesson header showing title and difficulty badge */}
      <header className="lesson-header">
        <h2>{lesson.title}</h2>
        <span className={`difficulty-badge difficulty-badge--
${lesson.difficulty.toLowerCase()}`}>
          {lesson.difficulty}
        </span>
      </header>

      {/* Rendered Markdown lesson content */}
      <div className="lesson-content">
        <ReactMarkdown>{lesson.markdownContent}</ReactMarkdown>
      </div>

      {/* Navigation to quiz for this lesson */}
      <footer className="lesson-footer">
```

```
        <a href={`/quiz/${lesson.quizId}`} className="btn btn--
primary">
          Take Quiz
        </a>
      </footer>

      {/* Floating "Ask AI" button — fixed position, always accessible
*/}
      <button
        className="ask-ai-fab"
        onClick={() => setAiPanelOpen(true)}
        aria-label="Ask AI a question about this lesson"
      >
        Ask AI
      </button>

      {/* Slide-in Q&A panel */}
      {aiPanelOpen && (
        <AskAiPanel
          topic={topic}
          onClose={() => setAiPanelOpen(false)}
        />
      )}
    </article>
  );
}
```

8.5 Quiz Interface and Score Summary

The quiz interface steps through questions one at a time and renders the score summary after submission.

```
// QuizInterface.jsx — Step-Through Quiz with Configurable Marks and
Score Summary
// Renders questions one at a time; tracks selected answers in
component state
// Submits all answers to the backend on final question; renders score
```

summary on completion

```
import { useState } from "react";
import axios from "../api/axiosInstance";
import { useQuiz } from "../hooks/useQuiz";
import ScoreSummary from "./ScoreSummary";

// QuizInterface controls the step-through quiz flow for the given
quizId
export default function QuizInterface({ quizId }) {
  const { quiz, loading } = useQuiz(quizId);
  const [currentIndex, setCurrentIndex] = useState(0);
  const [selectedAnswers, setSelectedAnswers] = useState([]);
  const [result, setResult] = useState(null);

  if (loading) return <div className="loader" aria-label="Loading
quiz..." />;
  if (!quiz) return <p>Quiz not found.</p>;

  const currentQuestion = quiz.questions[currentIndex];
  const isLastQuestion = currentIndex === quiz.questions.length - 1;

  // Records the learner's answer for the current question
  const handleAnswer = (optionIndex) => {
    const updated = [...selectedAnswers];
    updated[currentIndex] = optionIndex;
    setSelectedAnswers(updated);
  };

  // Advances to the next question or submits the quiz on the final
question
  const handleNext = async () => {
    if (!isLastQuestion) {
      setCurrentIndex((prev) => prev + 1);
      return;
    }
```

```
    // Submit answers to the backend scoring engine
    try {
      const res = await axios.post(`/quiz/${quizId}/submit`, {
        answers: selectedAnswers,
      });
      setResult(res.data); // Backend returns score, totalMarks, and
per-question breakdown
    } catch (err) {
      console.error("Quiz submission failed:", err);
    }
  };

  // Render score summary once results are received from the backend
  if (result) return <ScoreSummary result={result}
questions={quiz.questions} />;

  return (
    <section className="quiz-interface">
      {/* Progress indicator showing current question position */}
      <p className="quiz-progress" aria-label="Question progress">
        Question {currentIndex + 1} of {quiz.questions.length}
      </p>

      {/* Current question text and marks value */}
      <h3 className="question-text">{currentQuestion.text}</h3>
      <p className="question-marks">Marks: {currentQuestion.marks}</p>

      {/* Answer option buttons */}
      <ul className="options-list" role="radiogroup" aria-label="Answer
options">
        {currentQuestion.options.map((option, idx) => (
          <li key={idx}>
            <button
              className={`option-btn ${selectedAnswers[currentIndex]
=== idx ? "option-btn--selected" : ""}`}
              onClick={() => handleAnswer(idx)}
```

```
              aria-pressed={selectedAnswers[currentIndex] === idx}
            >
              {option}
            </button>
          </li>
        ))}
      </ul>

      {/* Navigation: Next Question or Submit Quiz */}
      <button
        className="btn btn--primary"
        onClick={handleNext}
        disabled={selectedAnswers[currentIndex] === undefined}
      >
        {isLastQuestion ? "Submit Quiz" : "Next Question"}
      </button>
    </section>
  );
}
```

8.6 Difficulty Badge and Shared Component Library

Shared UI components used across multiple feature modules are maintained in `src/components/shared/`. The `DifficultyBadge` component illustrates the approach taken for reusable presentational elements.

```
// DifficultyBadge.jsx — Reusable Difficulty Indicator Component
// Renders a color-coded badge reflecting the difficulty tier of a
lesson or quiz
// Used in GoalCard, LessonViewer header, and QuizInterface header
// Accepts a difficulty string prop; applies corresponding CSS class
for styling

// DifficultyBadge provides consistent visual treatment for difficulty
tiers
// across all feature modules without duplicating styling logic
export default function DifficultyBadge({ difficulty }) {
```

```
  // Normalize input to lowercase for CSS class consistency
  const level = difficulty ? difficulty.toLowerCase() : "medium";

  // Map difficulty levels to human-readable display labels
  const labelMap = {
    easy: "Easy",
    medium: "Medium",
    hard: "Hard",
    expert: "Expert",
  };

  return (
    <span
      className={`difficulty-badge difficulty-badge--${level}`}
      aria-label={`Difficulty: ${labelMap[level] || difficulty}`}
      role="img"
    >
      {labelMap[level] || difficulty}
    </span>
  );
}
```

Testing & Results

Frontend testing for the AI Adaptive Learning Platform was conducted systematically across authentication, goal management, lesson display, and quiz interaction.

9.1 Authentication UI Testing

TC	Test Case	Input	Expected Output	Result
TC01	Step 1 Screen Load	Navigate to /login	Credential form renders with email and password fields	Pass
TC02	Step 2 Screen Transition	Valid credentials submitted	OTP screen renders with countdown timer and 6-digit input	Pass
TC03	Inline Validation — Empty Email	Submit form with blank email	Inline error “Email is required” appears without page reload	Pass
TC04	OTP Expiry UI	Wait for countdown to reach zero	Countdown displays “00:00”; Resend OTP link activates	Pass
TC05	Successful Login Redirect	Correct OTP submitted	JWT stored; user redirected to /dashboard	Pass

Table 4: Authentication UI Test Cases and Results

9.2 Dashboard & Goal Creation Testing

TC	Test Case	Operation	Expected Output	Result
TC06	Dashboard Load	Navigate to /dashboard after login	All user goals rendered as cards; empty-state message if none exist	Pass
TC07	Goal Card Content	Existing goal with topic “Sun”, 5 days, Medium	Card shows topic, “Medium” difficulty badge, progress bar, timeframe	Pass
TC08	Goal Form Opens	Click “+ New Goal” button	Goal creation modal overlay appears with topic, timeframe, and difficulty inputs	Pass
TC09	Goal Created Successfully	Submit form with topic “Python”, 10 days, Hard	New goal card appears in dashboard grid; modal closes	Pass

Table 5: Dashboard and Goal Creation Test Cases and Results

9.3 Lesson Viewer & Quiz Interface Testing

TC	Test Case	Attempted Action	Expected Result	Result
TC10	Markdown Rendering	Open lesson with code blocks and headings	Code blocks syntax-highlighted; headings correctly hierarchical	Pass
TC11	Ask AI Panel Opens	Click floating "Ask AI" button	Q&A panel slides in without disrupting lesson scroll position	Pass
TC12	Quiz Step Navigation	Answer question 1 and click "Next Question"	Question 2 renders; progress indicator updates from 1/5 to 2/5	Pass
TC13	Answer Selection Highlight	Click an answer option	Selected button visually highlighted; other options remain unstyled	Pass
TC14	Score Summary Screen	Submit all answers in a 5-question quiz	Score summary renders with marks, percentage, and per-question feedback	Pass

Table 6: Lesson Viewer and Quiz Interface Test Cases and Results

All core frontend functionalities passed testing, confirming that the UI correctly renders dynamic content, handles user interactions, and communicates with the backend API across all primary user journeys.

Limitations

The following limitations apply to the current frontend implementation:

1. **Offline Support:** The current frontend requires a continuous internet connection. Lesson content and quiz questions are not cached locally, meaning learners in low-connectivity environments cannot access previously loaded content once the network is unavailable.
2. **Markdown Edge Cases:** While `react-markdown` handles standard Markdown correctly, complex nested structures such as tables within blockquotes or deeply nested lists may not render exactly as intended by the AI content generator.
3. **Real-time Streaming UI:** The "Ask AI" Q&A panel currently waits for the complete API response before rendering it. Progressive streaming display (token-by-token rendering) is planned but not yet implemented, resulting in a noticeable delay before the AI's answer appears.
4. **Bundle Size on Low-End Devices:** The inclusion of `react-markdown` and its syntax-highlighting dependency adds to the JavaScript bundle size. On very low-end mobile devices with limited memory, initial load time may be perceptible.
5. **No Persistent Local State:** If a learner closes the browser mid-quiz, their selected answers are lost because quiz state is held in React component memory. A session-recovery mechanism backed by `sessionStorage` is planned for a future release.
6. **Limited Accessibility Testing:** While basic WCAG 2.1 AA checks have been performed, a comprehensive audit with screen-reader software (NVDA, VoiceOver) has not yet been completed, and some complex interactive components may require additional ARIA attributes.

Conclusion & Future Scope

10.1 Conclusion

The AI Adaptive Learning Platform frontend represents a significant contribution to making AI-driven personalized education accessible and usable for a broad spectrum of learners. Built on React and Vite, the frontend delivers a responsive, component-based interface that translates sophisticated backend intelligence into a clean, navigable learner experience. The two-screen OTP authentication flow provides security without sacrificing usability — a balance critical for educational applications where diverse users, including younger students and non-technical adults, must authenticate reliably. The goal-creation interface democratizes access to personalized AI-generated curricula: any learner can define a learning objective in plain language and immediately receive a structured course tailored to their chosen timeframe and difficulty level.

The lesson viewer, powered by `react-markdown` and organized around a collapsible curriculum sidebar, presents AI-generated content in a format that supports sustained study sessions. The floating "Ask AI" button ensures that learners never need to interrupt their reading to seek clarification — a design choice informed by research showing that immediate, in-context support significantly improves comprehension and engagement. The step-through quiz interface, with its per-question answer highlighting and configurable marks display, provides a structured assessment experience that reinforces learning rather than merely evaluating it. The score summary screen closes the feedback loop by surfacing AI-generated explanations for incorrect responses, turning every wrong answer into a targeted learning moment.

The modular component architecture — organized by feature domain with co-located hooks and utilities — ensures that the frontend codebase remains maintainable and extensible as the platform evolves. The integration of ESLint, Prettier, and CI/CD pipelines embedded quality assurance into the development workflow from the outset, reducing defect escape rate and supporting confident iteration. This frontend implementation provides a scalable, production-ready user interface foundation upon which future enhancements — collaborative features, mobile applications, advanced analytics dashboards — can be built without requiring architectural overhaul.

10.2 Future Scope

1. Progressive Web App (PWA) Support: Converting the frontend to a Progressive Web App will enable offline lesson access, home-screen installation, and push notifications for goal reminders. Service workers will cache lesson content and quiz questions locally, allowing learners in low-connectivity regions to continue studying without interruption. Background sync will upload quiz results when the connection is restored. This enhancement is particularly impactful for learners in rural or underserved areas where reliable internet access cannot be assumed.

2. Real-Time Answer Streaming: Implementing token-by-token streaming for the "Ask AI" Q&A panel will dramatically reduce perceived response latency. Using the browser's `EventSource` API or WebSocket connections, the panel will begin rendering the AI's response as individual tokens arrive, creating a conversational, fluid interaction pattern that mirrors typing — a familiar and reassuring experience for learners accustomed to modern chat interfaces.

3. Advanced Progress Visualization: Integrating D3.js or Chart.js to build an interactive analytics panel will allow learners to visualize their score progression over time, identify weak topics through heatmaps, and view learning velocity trends. Progress rings, streak calendars, and topic-mastery radar charts will transform raw performance data into actionable insights that motivate continued study. Educators in institutional deployments will access cohort-level views for curriculum effectiveness analysis.

4. Gamification Layer: Introducing a badge and achievement system rendered in the dashboard will incentivize consistent engagement. Learners will earn visual badges for milestones such as completing ten quizzes, maintaining a seven-day study streak, or achieving 90%+ on a Hard-difficulty assessment. An optional leaderboard component — scoped to classroom cohorts to encourage healthy competition — will reinforce social accountability. These features are grounded in behavioral psychology research demonstrating that visible progress markers significantly increase platform retention rates.

5. Native Mobile Application: Developing a React Native counterpart to the web frontend will extend platform accessibility to learners who primarily use smartphones. The shared component logic and custom hooks can be adapted for React Native with

minimal duplication. Mobile-specific enhancements will include swipe-to-advance quiz navigation, haptic feedback on correct answers, and a compact lesson viewer optimized for portrait-mode reading on small screens.

6. Collaborative Learning UI: Adding discussion threads anchored to specific lesson paragraphs will enable peer-to-peer knowledge sharing directly within the lesson viewer. Learners will be able to highlight a passage and open a threaded comment panel, creating a social annotation layer on top of AI-generated content. Study-group dashboards will display shared goals, combined progress metrics, and a joint quiz-challenge interface — fostering community-based learning alongside the existing individualized experience.

7. Adaptive Difficulty Progression UI: Once the backend implements automatic difficulty adjustment based on quiz performance trends, the frontend will surface this progression through animated difficulty-tier transitions on the goal card. Learners who consistently exceed the target score threshold will see their difficulty badge animate from "Medium" to "Hard," accompanied by a congratulatory micro-interaction. Conversely, learners who fall below the threshold will receive a gentle nudge suggesting a review session — communicated through a non-intrusive toast notification rather than a disruptive modal.

8. Spaced Repetition Review Queue: A "Review" tab on the dashboard will surface lessons and quiz topics due for spaced repetition based on the SM-2 scheduling algorithm. Learners will see a queue of items to review each day, presented in order of urgency (most overdue first). Completing the review queue will update a retention health score displayed prominently on the dashboard, giving learners a tangible measure of how well they are retaining previously studied material.

9. Internationalization (i18n) Support: Integrating `react-i18next` will enable the UI to be rendered in multiple languages without requiring code changes. Language files will be maintained as JSON key-value stores, allowing non-developer contributors to add or update translations. Right-to-left language support (Arabic, Hebrew, Urdu) will be implemented through dynamic document direction switching. This enhancement is critical for the platform's goal of serving learners across diverse linguistic communities.

10. Accessibility Deep Audit and Remediation: A comprehensive accessibility audit using NVDA and VoiceOver screen readers, supplemented by automated checks with Axe-core integrated into the CI pipeline, will systematically identify and remediate any remaining WCAG 2.1 AA gaps. Focus management during modal transitions, ARIA live region announcements for dynamic content updates, and keyboard-accessible custom dropdowns will be prioritized. This effort will ensure the platform is genuinely usable by learners with visual impairments, motor disabilities, or cognitive differences — aligning with the platform's inclusive education mission.

11. Dark Mode and Theming: Implementing a CSS custom-property-based theming system will allow learners to switch between light and dark color schemes according to their preference. The theme selection will persist across sessions via `localStorage` and will automatically respect the operating system's preferred color scheme via the `prefers-color-scheme` media query. Additional contrast themes optimized for learners with low vision will be offered in the accessibility settings panel, ensuring no learner is excluded by display preferences.

12. Component Library Publication: Extracting the shared component library — including `DifficultyBadge`, `ProgressBar`, `GoalCard`, and the `AskAiPanel` — into a standalone npm package will allow other educational applications within the institution or organization to adopt a consistent visual language. The library will be documented using Storybook, enabling designers and developers to browse, test, and integrate components in isolation without spinning up the full application. This promotes design consistency and accelerates UI development for future platform extensions.

13. Real-Time Collaboration Features: Integration with WebSocket-based co-presence tooling will allow multiple learners to study the same lesson simultaneously in a shared session. Presence indicators will show which peers are currently reading each lesson, and a shared cursor annotation tool will enable real-time collaborative highlighting and note-taking. This transforms the currently single-player learning experience into a socially enriched environment that mirrors the collaborative dynamics of physical study groups.

14. AI-Powered Lesson Personalization Controls: Exposing learner-controlled personalization parameters in the lesson viewer — such as "Explain with code examples," "Use analogies," or "Give me the short version" — will allow learners to reshape AI-

generated content to match their learning style without re-creating their goal. These preferences will be passed to the backend as additional prompt parameters, allowing the AI to regenerate or supplement the current lesson content on demand. This level of granular control represents the next frontier of truly personalized AI-driven education.

15. Institutional Admin Dashboard: For schools and coaching centers deploying the platform at scale, a dedicated administrator interface will provide cohort-level insights: aggregate score distributions, topic mastery heatmaps, at-risk student identification, and curriculum effectiveness analysis. Bulk user management, custom institutional branding, and exportable reports in CSV and PDF formats will support accreditation documentation and compliance requirements. Role-based access control will ensure that administrators can view cohort data without accessing individual learner records.

Bibliography

[1] OpenAI. **OpenAI API Documentation**. Available at: https://platform.openai.com/docs. Accessed May 2026.

[2] MongoDB, Inc. **MongoDB Documentation**. Available at: https://www.mongodb.com/docs. Accessed May 2026.

[3] Node.js Foundation. **Node.js Documentation**. Available at: https://nodejs.org/docs. Accessed May 2026.

[4] React Community. **React Documentation**. Available at: https://react.dev. Accessed May 2026.

[5] Vite. **Vite Documentation**. Available at: https://vitejs.dev. Accessed May 2026.

[6] Pressman, Roger S. **Software Engineering: A Practitioner's Approach**, 8th ed. McGraw-Hill Education, 2014.

[7] Axios. **Axios HTTP Client Documentation**. Available at: https://axios-http.com/docs. Accessed May 2026.

APPENDIX A: CODING

This appendix contains core code snippets from the frontend components of the AI Adaptive Learning Platform.

A.1 Custom Hook: useGoals

```
// hooks/useGoals.js
// useGoals — Custom React Hook for Goal Lifecycle Management
// Abstracts all API calls related to goal fetching, creation, and
status updates
// Provides loading and error states to consuming components
// Designed to keep Dashboard.jsx and GoalCard.jsx focused on rendering
only

import { useState, useEffect } from "react";
import axios from "../api/axiosInstance";

// useGoals encapsulates the full goal data lifecycle for the dashboard
// Returns: goals array, loading flag, error message, and createGoal
function
const useGoals = () => {
  // goals: array of goal objects fetched from the backend
  const [goals, setGoals] = useState([]);

  // loading: true while any async operation is in progress
  const [loading, setLoading] = useState(true);

  // error: string containing the last error message, or null if none
  const [error, setError] = useState(null);

  // Fetch all goals for the authenticated user on component mount
  // Re-runs whenever the component mounts (empty dependency array)
  useEffect(() => {
    const fetchGoals = async () => {
      try {
```

```
        const res = await axios.get("/goals");
        setGoals(res.data); // Backend returns array of goal documents
      } catch (err) {
        setError(err.response?.data?.error || "Failed to load goals.");
      } finally {
        setLoading(false);
      }
    };
    fetchGoals();
  }, []);

  // Creates a new goal via POST request and appends it to the goals
array
  // Accepts goalData: { topic, timeframe, difficulty }
  const createGoal = async (goalData) => {
    setLoading(true);
    try {
      const res = await axios.post("/goals", goalData);
      setGoals((prev) => [...prev, res.data]); // Append new goal
optimistically
    } catch (err) {
      setError(err.response?.data?.error || "Failed to create goal.");
    } finally {
      setLoading(false);
    }
  };

  // Index for efficient querying
  // Returns state and actions for consumer components
  return { goals, loading, error, createGoal };
};

module.exports = useGoals;
```

A.2 Axios Interceptor Configuration

```
// api/axiosInstance.js
// Configured Axios instance with JWT injection and 401 handling
import axios from "axios";

const instance = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost:5000/
api/v1",
});

// Attach JWT token to every outgoing request
instance.interceptors.request.use((config) => {
  const token = sessionStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Redirect to login on 401 Unauthorized responses
instance.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      sessionStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);

export default instance;
```

APPENDIX B: ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
HMR	Hot Module Replacement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MERN	MongoDB, Express, React, Node.js
NoSQL	Not Only SQL
OTP	One-Time Password
PWA	Progressive Web Application
REST	Representational State Transfer
UI	User Interface
WCAG	Web Content Accessibility Guidelines

Table 7: Abbreviations and Acronyms

APPENDIX C: FRONTEND COMPONENT TREE

This appendix documents the hierarchical component structure of the AI Adaptive Learning Platform frontend, organized by feature domain. Each entry describes the component's purpose and its parent-child relationship within the React component tree.

C.1 Application Root

```
App (src/App.jsx)
├── Router (React Router v6 BrowserRouter)
│   ├── PrivateRoute (src/components/shared/PrivateRoute.jsx)
│   │   └── [Protected pages — rendered only when JWT is present]
│   ├── /login       → AuthPage
│   ├── /dashboard   → Dashboard
│   ├── /lesson/:id  → LessonViewer
│   └── /quiz/:id    → QuizInterface
```

The `PrivateRoute` wrapper component intercepts navigation to protected paths. If no valid JWT is found in `sessionStorage`, it immediately redirects the user to `/login`, preventing unauthorized access to lesson and quiz content.

C.2 Authentication Feature

```
AuthPage (src/features/auth/AuthPage.jsx)
├── CredentialStep (src/features/auth/CredentialStep.jsx)
│   ├── EmailInput (src/components/shared/EmailInput.jsx)
│   ├── PasswordInput (src/components/shared/PasswordInput.jsx)
│   └── SubmitButton (src/components/shared/SubmitButton.jsx)
└── OtpStep (src/features/auth/OtpStep.jsx)
    ├── OtpInput (src/features/auth/OtpInput.jsx)
    ├── CountdownTimer (src/features/auth/CountdownTimer.jsx)
    └── ResendOtpLink (src/features/auth/ResendOtpLink.jsx)
```

`CredentialStep` and `OtpStep` are rendered conditionally based on the `step` state variable held in `AuthPage`. The `CountdownTimer` component uses `useEffect` with a 1-sec-

ond interval to decrement the displayed remaining time, activating the `ResendOtpLink` when the countdown reaches zero.

C.3 Dashboard Feature

```
Dashboard (src/features/dashboard/Dashboard.jsx)
├── GoalCard[] (src/features/dashboard/GoalCard.jsx)
│   ├── DifficultyBadge (src/components/shared/DifficultyBadge.jsx)
│   ├── ProgressBar (src/components/shared/ProgressBar.jsx)
│   └── GoalActionButtons (src/features/dashboard/
GoalActionButtons.jsx)
├── GoalForm (src/features/dashboard/GoalForm.jsx)
│   ├── TopicInput (src/features/dashboard/TopicInput.jsx)
│   ├── TimeframeDropdown (src/features/dashboard/
TimeframeDropdown.jsx)
│   └── DifficultyRadioGroup (src/features/dashboard/
DifficultyRadioGroup.jsx)
└── FloatingActionButton (src/components/shared/
FloatingActionButton.jsx)
```

`GoalCard` is a pure presentational component — it receives a `goal` prop and renders it without making API calls. All data fetching and mutation logic is encapsulated in the `useGoals` custom hook consumed by `Dashboard`. The `GoalForm` is rendered inside a portal (`ReactDOM.createPortal`) to ensure it overlays the full viewport regardless of its position in the component tree.

C.4 Lesson Viewer Feature

```
LessonViewer (src/features/lessons/LessonViewer.jsx)
├── LessonSidebar (src/features/lessons/LessonSidebar.jsx)
│   └── LessonNavItem[] (src/features/lessons/LessonNavItem.jsx)
├── LessonContent (src/features/lessons/LessonContent.jsx)
│   └── ReactMarkdown (react-markdown library)
│       └── SyntaxHighlighter (react-syntax-highlighter library)
├── LessonFooter (src/features/lessons/LessonFooter.jsx)
│   └── StartQuizButton (src/components/shared/SubmitButton.jsx)
└── AskAiPanel (src/features/lessons/AskAiPanel.jsx)
```

```
    ├── AskAiInput (src/features/lessons/AskAiInput.jsx)
    └── AiResponseDisplay (src/features/lessons/AiResponseDisplay.jsx)
```

`LessonSidebar` receives the full list of lessons for the current goal and renders a navigation item per lesson. Clicking a `LessonNavItem` updates the active lesson ID in the parent `LessonViewer` without a full page reload, enabling smooth in-goal navigation. `AskAiPanel` is conditionally rendered based on the `aiPanelOpen` boolean state in `LessonViewer` and slides in via a CSS transition.

C.5 Quiz Feature

```
QuizInterface (src/features/quiz/QuizInterface.jsx)
├── QuizProgress (src/features/quiz/QuizProgress.jsx)
├── QuestionCard (src/features/quiz/QuestionCard.jsx)
│   ├── QuestionText (src/features/quiz/QuestionText.jsx)
│   ├── MarksLabel (src/features/quiz/MarksLabel.jsx)
│   └── OptionButton[] (src/features/quiz/OptionButton.jsx)
├── NavigationButton (src/components/shared/SubmitButton.jsx)
└── ScoreSummary (src/features/quiz/ScoreSummary.jsx)
    ├── ScoreHeader (src/features/quiz/ScoreHeader.jsx)
    └── QuestionFeedbackItem[] (src/features/quiz/
QuestionFeedbackItem.jsx)
        └── DifficultyBadge (src/components/shared/DifficultyBadge.jsx)
```

`QuizInterface` manages the full quiz state in a single component: `currentIndex`, `selectedAnswers` array, and `result` object. `ScoreSummary` is rendered in place of the quiz interface once the `result` state is populated following a successful API submission. `QuestionFeedbackItem` expands on click to reveal the AI-generated explanation for each question, reducing visual clutter on the initial summary view.

The component tree described above reflects the state of the codebase at the end of Sprint 5, representing a carefully architected snapshot of the platform's frontend structure at a specific point in time. This particular iteration of the component tree embodies design decisions made throughout the first five sprints of development, incorporating lessons learned from initial prototyping, user feedback, and iterative refinement cycles. The tree structure itself is not static but rather a living artifact that evolves in response to

changing requirements, emerging architectural insights, and the ongoing maturation of the codebase. Understanding the current state of the component tree is essential for comprehending how the platform is organized and how future enhancements will be integrated into the existing structure.

The organization of components at the end of Sprint 5 reflects a deliberate architectural choice to group related functionality into cohesive modules rather than organizing by generic type or layer. Each major feature area — such as authentication, content delivery, assessment, and user profile management — maintains its own subtree of components, hooks, and utility functions. This approach, often referred to as feature-based or domain-driven organization, contrasts with older architectural patterns that would separate all components into a single folder, all hooks into another, and all utilities into yet another. The feature-based approach has proven superior in terms of developer experience, code discoverability, and the ability to reason about feature boundaries and dependencies.

Future iterations of the platform will introduce significant new capabilities that will expand the component tree in strategic directions. The analytics dashboard represents one of the most substantial planned additions, requiring a new subtree of visualization components, data fetching hooks, and utility functions for aggregating and transforming raw event data into meaningful insights. This dashboard will serve educators and institutional administrators, providing them with real-time views of student engagement, performance trends, and system health metrics. The components required for this feature will follow the same organizational principles established in earlier sprints, ensuring consistency and maintainability across the codebase.

Collaborative study features constitute another major enhancement planned for future sprints, introducing entirely new interaction patterns and data synchronization requirements. These features will enable students to form study groups, share notes, engage in peer review, and collaborate on problem-solving exercises in real-time. The component tree will need to expand to accommodate real-time collaboration components, WebSocket-based communication hooks, conflict resolution utilities, and presence awareness systems. The architectural decisions made regarding how these components interact with existing state management and data fetching patterns will be critical to ensuring that the collaborative features integrate seamlessly rather than creating isolated silos within the codebase.

The PWA service worker integration layer represents a third major addition that will reshape how the frontend architecture handles offline functionality and background synchronization. Service workers operate in a different execution context than the main JavaScript thread, requiring careful coordination through message passing and shared state mechanisms. The component tree will need to expand to include service worker registration components, offline state management hooks, and utilities for queuing and retrying failed network requests when connectivity is restored. This integration will enable the platform to function gracefully in low-connectivity environments, a critical requirement for institutional deployments in regions with unreliable internet infrastructure.

The deliberate co-location of each feature's components, hooks, and utilities represents a fundamental architectural principle that has guided the organization of the codebase since its inception. Rather than forcing developers to navigate across multiple directories to understand how a feature works, all related code is grouped together in a single feature folder. This approach dramatically reduces cognitive load when working on a feature, as all the necessary context is immediately available without requiring extensive cross-directory navigation. A developer tasked with implementing a new feature or fixing a bug can locate all relevant code within a single subtree, understand the dependencies and interactions, and make changes with confidence that they understand the full scope of impact.

This co-location principle extends beyond mere file organization and reflects a deeper commitment to modularity and separation of concerns at the architectural level. Each feature module maintains clear boundaries, with explicit interfaces for how it interacts with other modules. Dependencies flow in predictable directions, making it easy to understand the impact of changes and to identify potential circular dependencies or architectural violations. The principle also facilitates testing, as each feature module can be tested in isolation or in combination with other modules in a controlled manner. This modular approach has proven invaluable as the codebase has grown, preventing the common problem of "spaghetti code" where dependencies become tangled and difficult to reason about.

As the platform scales and new features are added, the component tree will inevitably grow in complexity and size. However, the architectural principles established in the first five sprints provide a clear framework for managing this growth in a controlled

manner. New features will be added as new subtrees, following the same organizational patterns and naming conventions as existing features. The co-location principle ensures that each new feature remains self-contained and maintainable, even as the overall codebase expands. This approach has been proven effective in large-scale frontend applications, where codebases can contain thousands of components organized across hundreds of feature modules without becoming unmaintainable.

The maintainability of the component tree is further enhanced by the consistent application of architectural patterns and conventions across all feature modules. Naming conventions for components, hooks, and utilities are standardized, making it easy for new developers to understand the purpose and scope of any piece of code they encounter. Documentation is co-located with code, ensuring that architectural decisions and usage patterns are captured in proximity to the implementation. Code review processes enforce adherence to these patterns, preventing drift and ensuring that the codebase remains coherent as multiple developers contribute changes across different features and sprints.

Looking forward, the component tree will continue to evolve in response to new requirements, emerging best practices, and lessons learned from operational experience. The analytics dashboard, collaborative study features, and PWA service worker integration represent the next major milestones in this evolution, but they are not the final destination. The architecture has been designed with sufficient flexibility to accommodate future enhancements without requiring wholesale restructuring. The principles of feature-based organization, co-location of related code, and clear separation of concerns provide a solid foundation upon which the platform can grow and mature over time.

In summary, the component tree at the end of Sprint 5 represents a well-architected, maintainable, and scalable foundation for the AI Adaptive Learning Platform's frontend. The deliberate choices made regarding code organization, feature boundaries, and architectural patterns have created a codebase that is easy to navigate, understand, and extend. As future iterations introduce new capabilities and expand the feature set, the same principles will guide the integration of new components and the evolution of the overall architecture. This commitment to architectural discipline and consistency ensures that the platform can scale effectively without sacrificing code quality, maintainability, or developer experience.